

Group Members: Kasra Mokhtari, Jacob Wei, Andrei Ionov

For our project, we implemented a text spam detector that classifies messages and/or emails as either spam or ham (non-spam). Our approach to creating the spam detector was to take raw text from messages or emails, turn them into numerical features, and then train a machine learning model to learn the differences between spam and non-spam messages. For training our models, we decided to use the SMS Spam Collection dataset for text messages and the Enron spam dataset for emails, which let us test the same machine learning approach on two different types of text.

The general flow of our project was that we loaded the data, cleaned the text, converted it into TF-IDF features, trained the models, and evaluated the results. For the SMS dataset, each row of data already had a label and a message together. For the Enron dataset, we decided to combine the subject line with the message itself into one text field since both can be useful for determining whether an email is spam or not. During preprocessing, we decided to set the text to all lowercase letters, and replaced text elements like URLs, numbers, and email addresses with general tokens. This helped the model learn general patterns, like the existence of a link or number in the text, instead of memorizing specific URLs, phone numbers, or email addresses from the training data.

The main model we implemented was TF-IDF with a Linear Support Vector Machine. We chose TF-IDF because it is a simple and easily understandable way to represent text. TF-IDF gives more weight to words or phrases that are important in a message rather than words that are just common across many messages. We chose a Linear SVM because TF-IDF creates a large number of word and phrase features, but each message only uses a small subset of them. Linear

models work well in this area of text data because they can learn which features push a message toward spam or non-spam without a more complex model. We also used

`class_weight="balanced"` in the implementation because the SMS dataset had many more non-spam messages than spam, so doing so allowed the model to give more weight to the smaller spam class during training, which helped reduce any bias the model might have had toward the larger non-spam class.

For the structure of our project, we used both a Jupyter notebook and Python helper files to create this project. We had one Jupyter notebook, `spam_detection_final.ipynb`, which contained the main explanation, experiments, tables, and graphs of the project. The helper files were modularized to handle things like loading the data, cleaning the text, building the models, calculating the results, creating some of the visualizations, and explaining the predictions the model made. This allowed us to keep the notebook clutter-free and easy to follow.

For the testing portion of our project, we used a standard train/test split along with 5-fold cross-validation. We used stratified splitting so that the spam and non-spam balance stayed fairly consistent between the training and testing sets. For cross-validation, we also used stratified shuffled folds, which kept each fold balanced while also preventing the results from depending too much on the original order of the dataset. To evaluate the models, we used accuracy, precision, recall, F1, and F2 because spam detection has two important types of mistakes: incorrectly labeling a real message as spam and incorrectly allowing a spam message to be labeled as non-spam. Both mistakes matter, so we did not want to rely only on accuracy.

One issue we noticed was that our first cross-validation results for the Enron dataset were inconsistent, even though the normal train/test split looked extremely strong. After looking into

it, we realized that the dataset order could affect the folds if the data was not shuffled. To fix this, we changed cross-validation to use stratified shuffled folds, which made the results much more stable. Another challenge was that the SMS dataset was imbalanced, with many more non-spam messages than spam messages. Because of this, accuracy alone could be misleading, so we used `class_weight="balanced"` for the Linear SVM and focused more on precision, recall, F1, and F2.

We created two baseline models, Naive Bayes and Logistic Regression, to compare against our Linear SVM. On the SMS dataset, the Linear SVM performed well, with about 0.987 accuracy, 0.941 precision, 0.960 recall, 0.950 F1, and 0.956 F2. Naive Bayes had perfect precision on the SMS test split but lower recall, meaning it was careful when marking spam but missed more spam messages. Logistic Regression had high recall but lower precision. Based on the results, the Linear SVM had the best balance and overall performance. On the Enron dataset, the Linear SVM and Logistic Regression both performed very well, yielding results around 0.999 accuracy and almost perfect recall.

To do further testing and experimentation, we added an n-gram experiment. Our original model used word n-grams with $n=1$ and $n=2$, so it looked at individual words and two-word phrases specifically. We then tested character n-grams with $n=3$ to $n=5$, so our models could look at character patterns that were inside words that are commonly used in spam messages, like “pr!ze,” “cl!ck,” or “fr33.” At the end, we combined the word and character model and tested it. We found that for SMS, the combined model performed slightly better than before, with about 0.991 accuracy and 0.958 F2, which showed us that character-level patterns can give the model a little more information to make more accurate predictions.

The final part of our project was a Streamlit demo app. The app loads the saved SMS model, lets a user paste in a message, and predicts whether the message is spam or non-spam. We also wanted the app to do more than just give a label, so we added an interpretability feature that shows the top model signals behind the prediction. Since the SVM is linear, we can look at the feature weights it learned during training. Features with positive weights push the model toward spam, while features with negative weights push it toward non-spam. For each user message, the app shows which words or phrases contributed most to the prediction, making the result easier to understand. This connected our notebook experiments to a simple usable interface while also showing some of the evidence behind the model's decision.

Overall, our project showed that TF-IDF with a Linear SVM is a strong and understandable approach for spam detection. The model performed well on both SMS and email data, and the baseline comparisons, n-gram experiment, cross-validation, confusion matrices, and interpretability section helped us understand the model more in-depth. The main limitation we saw was that both datasets are older, and modern spam or phishing messages may be harder to detect. In the future, we could test on newer datasets and add metadata features like sender information or links.